

Universidad de Costa Rica

Laboratorio de Microcontroladores

Laboratorio #2:
GPIOs, Timers y FSM

Prof. MSc. Marco Villalta Fallas

Marco Vásquez Ovares
B17032
Grupo: 01

I Ciclo 2025

Índice

1. Introducción	6
2. Nota teórica	7
2.1. Características generales	8
2.2. Diagrama de pines	9
2.3. Descripción de pines	10
2.3.1. Funciones alternativas del puerto A	10
2.3.2. Funciones alternativas del puerto B	10
2.3.3. Funciones alternativas del puerto D	11
2.4. Diagrama de bloques	12
2.5. Mapa de memoria de datos	13
2.6. Características eléctricas	13
2.6.1. Absolute Maximum Ratings	13
2.6.2. DC Characteristics	14
2.7. Periféricos	14
2.7.1. Registros	14
2.7.2. Registros de puertos (I/O)	15
2.7.3. Registros de interrupciones	15
2.7.4. Registros del temporizador	16
2.8. Componentes electrónicos complementarios	17
2.8.1. Leds	17
2.8.2. Botón con configuración Pull-Down	19
2.9. Diseño del circuito	19
2.9.1. Lista de componentes	20
2.9.2. Información adicional	20
3. Desarrollo	23
3.1. Análisis del firmware	23
3.1.1. Configuración General y Variables	23
3.1.2. Definición de Estados para la FSM	23
3.1.3. Función principal (main)	24
3.1.4. Función principal del semáforo	24
3.1.5. Máquina de estados finita (FSM)	24
3.1.6. Estado <code>paso_vehiculos</code> (vehículos en verde)	25
3.1.7. Estado <code>waiting</code> (vehículos en amarillo)	25
3.1.8. Estado <code>paso_peatones</code> (peatones en verde)	25
3.1.9. Función de retardo con temporizador	25
3.1.10. Interrupción por botón (ISR externa)	26
3.1.11. Interrupción por Timer Overflow	26
3.1.12. Cálculo de Overflows	26
3.2. Análisis electrónico	27
3.2.1. Implementación de los botones	27
3.2.2. Iluminación de los Leds	29
3.2.3. Led rojo	29
3.2.4. Led amarillo	30

3.2.5. Led verde	31
3.2.6. Configuración del reloj en el MCU	31
3.3. Análisis de resultados	32
3.3.1. Paso de vehículos	33
3.3.2. Waiting (Modo de espera)	33
3.3.3. Paso de peatones	34
4. Conclusiones y Recomendaciones	36
4.1. Conclusiones	36
4.2. Recomendaciones	36
Bibliografía	37
5. Apéndices	37
5.1. Repositorio Git	37

Índice de figuras

1.	Microcontrolador ATtiny4313 (SOIC/20)	8
2.	Microcontrolador ATtiny4313 (PDIP/20)	9
3.	Diagrama de Pines, Microcontrolador ATtiny4313	9
4.	Alternate Functions of Port A	10
5.	Alternate Functions of Port B	10
6.	Alternate Functions of Port B	10
7.	Alternate Functions of Port D	11
8.	Diagrama de bloques, Microcontrolador ATtiny4313	12
9.	AVR CPU General Purpose Working Registers, Microcontrolador ATtiny4313	13
10.	Absolute Maximum Ratings, Microcontrolador ATtiny4313	13
11.	DC Characteristics, Microcontrolador ATtiny4313	14
12.	DC Characteristics, Microcontrolador ATtiny4313	14
13.	Port Pin Configurations	15
14.	GIMSK – General Interrupt Mask Register	15
15.	PCMSK0 – Pin Change Mask Register 0	15
16.	TCCR0A – Timer/Counter Control Register A	16
17.	TCCR0B – Timer/Counter Control Register B	16
18.	Clock Select Bit Description	16
19.	TCNT0 – Timer/Counter Register	17
20.	TIMSK – Timer/Counter Interrupt Mask Register	17
21.	Diseño final, parte 1	19
22.	Diseño final, parte 2	20
23.	Diseño final, parte 3	20
24.	Reset and Interrupt Vectors	21
25.	Diagrama de estados de la FSM.	22
26.	Botón off	27
27.	Botón1 on	28
28.	Botón2 on	28
29.	Datasheet led rojo	29
30.	Led rojo	29
31.	Datasheet led amarillo	30
32.	Led amarillo	30
33.	Datasheet led verde	31
34.	Led verde	31
35.	Configuración del reloj	32
36.	Funcionamiento estado paso de vehículos	33
37.	Características de los componentes, estado paso de vehículos	33
38.	Funcionamiento estado waiting	34
39.	Características de los componentes, estado waiting	34
40.	Funcionamiento estado paso de peatones	35
41.	Características de los componentes, estado paso de peatones	35

Índice de tablas

1. Lista de componentes 21

1. Introducción

Resumen

En este laboratorio se desarrolla un sistema de control de semáforos vehicular y peatonal utilizando el microcontrolador **ATtiny4313**, haciendo uso de interrupciones externas y temporización basada en el temporizador interno del microcontrolador. El objetivo principal fue simular un cruce peatonal controlado por botones que interrumpen el flujo vehicular permitiendo el paso seguro de peatones.

El sistema implementa una máquina de estados finita para gestionar las transiciones entre el paso de vehículos, la espera y el paso de peatones. Se configuran los pines del microcontrolador para manejar LEDs que representan los distintos estados de los semáforos, y botones que generan interrupciones al ser presionados. Para el control de los tiempos de encendido y apagado de los LEDs, se utiliza el **Timer0** en modo normal con interrupción por desbordamiento, lo que permite simular de forma precisa los intervalos de tiempo, 3s y 10s, requeridos por el sistema.

Durante el desarrollo se verifica el correcto funcionamiento de los botones mediante interrupciones por cambio de pin (**PCINT**), y se diseña una lógica que desactiva estas interrupciones para evitar múltiples lecturas erróneas. También se comprueba que los tiempos de espera para el cruce peatonal y la advertencia a los vehículos se cumplen conforme al diagrama de temporización provisto en el enunciado.

Como resultado, se logra implementar un sistema funcional y eficiente que responden a las solicitudes peatonales y respeta los tiempos definidos, todo ello mediante una arquitectura de programación estructurada en interrupciones y máquinas de estados finita.

2. Nota teórica

El ATtiny4313 es un microcontrolador de 8 bits perteneciente a la familia AVR de Microchip (anteriormente Atmel). Es un dispositivo compacto pero versátil, ideal para aplicaciones embebidas que requieren bajo consumo y un número reducido de periféricos. A continuación, se describen sus características más relevantes.

Características principales:

- Arquitectura AVR de 8 bits tipo RISC: ejecuta instrucciones en un solo ciclo de reloj, lo que permite alto rendimiento con bajo consumo.
- Velocidad de operación: puede funcionar hasta 20 MHz con oscilador externo, o hasta 8 MHz utilizando el oscilador interno calibrado.
- Memoria:
 - 4 KB de memoria Flash (para almacenamiento del programa).
 - 256 bytes de memoria EEPROM (para datos no volátiles).
 - 256 bytes de SRAM (memoria de datos en tiempo de ejecución).
- Número de pines: encapsulado de 20 pines (PDIP, SOIC, o TSSOP).
- Entradas/Salidas (I/O): hasta 18 líneas de entrada/salida configurables.
- Timers/Contadores:
 - Un temporizador de 8 bits con modos normal, CTC y PWM.
 - Un temporizador de 16 bits.
- Interrupciones: admite interrupciones externas (INT0, INT1) y por cambio de pin (PCINT), además de interrupciones internas como desbordamientos de temporizadores.
- Comunicación:
 - USART de alta velocidad.
 - Interfaz SPI.
- Otros periféricos:
 - Comparador analógico.
 - Watchdog Timer.
 - Oscilador interno con opción de auto-calibración.

SOIC / 20

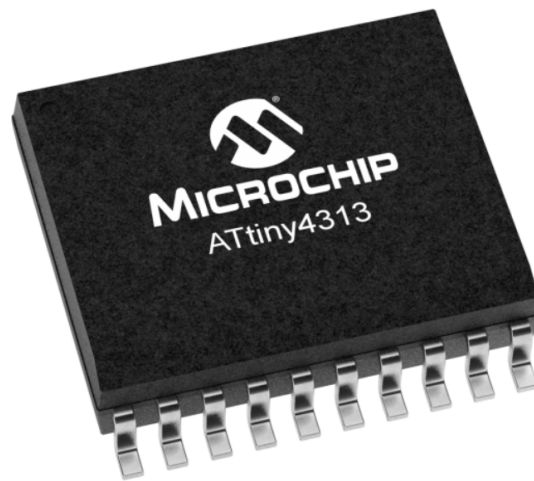


Figura 1: Microcontrolador ATtiny4313 (SOIC/20)

2.1. Características generales

El ATtiny4313 es un microcontrolador de la familia AVR, desarrollado por Microchip. Es un chip compacto de 20 pines que ofrece una buena combinación de rendimiento y simplicidad, ideal para aplicaciones pequeñas donde se requiere control básico, lectura de entradas digitales y manejo de señales.

Entre sus principales características generales destacan:

- Arquitectura: AVR de 8 bits, basada en un conjunto de instrucciones tipo RISC.
- Frecuencia de operación: hasta 20 MHz con oscilador externo o 8 MHz usando el oscilador interno.
- Memoria:
 - 4 KB de memoria Flash para almacenar el programa.
 - 256 bytes de SRAM.
 - 256 bytes de EEPROM.
- Entradas/Salidas digitales: hasta 18 pines configurables como entrada o salida.
- Temporizadores:
 - Un temporizador de 8 bits (Timer0).
 - Un temporizador de 16 bits (Timer1).
- Interrupciones: manejo de interrupciones por pines, temporizadores y eventos internos.
- Comunicación serial: incluye soporte para UART/USART y SPI.

- Alimentación: funciona con tensiones entre 2.7V y 5.5V.
- Consumo de energía: bajo consumo en modos de reposo y operación normal,
- Encapsulado: disponible en versiones DIP (20 pines), SOIC y TSSOP.

PDIP / 20

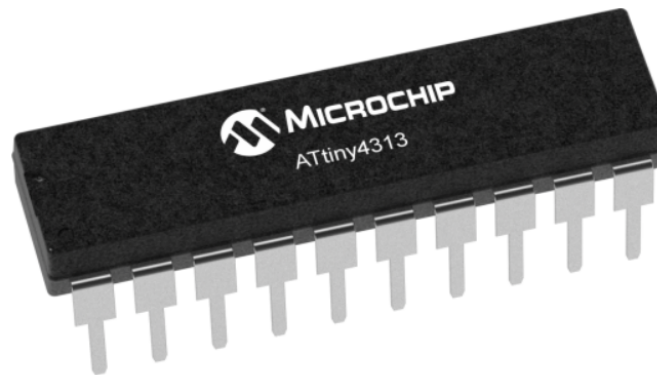


Figura 2: Microcontrolador ATtiny4313 (PDIP/20)

2.2. Diagrama de pines

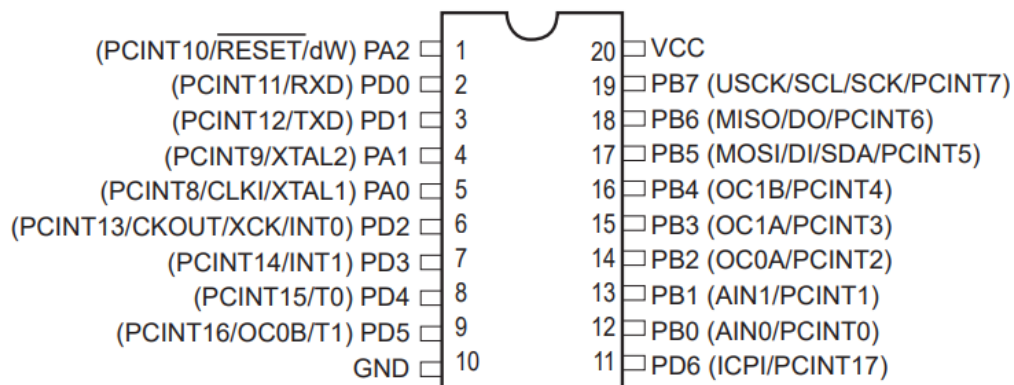


Figura 3: Diagrama de Pines, Microcontrolador ATtiny4313

2.3. Descripción de pines

2.3.1. Funciones alternativas del puerto A

Port Pin	Alternate Function
PA0	XTAL1: Crystal Oscillator Input CLKI: External Clock Input PCINT8: Pin Change Interrupt 1, Source 8
PA1	XTAL2: Crystal Oscillator Output PCINT9: Pin Change Interrupt 1, Source 9
PA2	RESET: Reset pin dW: debugWire I/O PCINT10: Pin Change Interrupt 1, Source 10

Figura 4: Alternate Functions of Port A

2.3.2. Funciones alternativas del puerto B

Port Pin	Alternate Function
PB0	AIN0: Analog Comparator, Positive Input PCINT0: Pin Change Interrupt 0, Source 0
PB1	AIN1: Analog Comparator, Negative Input PCINT1: Pin Change Interrupt 0, Source 1
PB2	OC0A: Timer/Counter0 Compare Match A Output PCINT2: Pin Change Interrupt 0, Source 2
PB3	OC1A: Timer/Counter1 Compare Match A Output PCINT3: Pin Change Interrupt 0, Source 3

Figura 5: Alternate Functions of Port B

Port Pin	Alternate Function
PB4	OC1B: Timer/Counter1 Compare Match B Output PCINT4: Pin Change Interrupt 0, Source 4
PB5	DI: USI Data Input (Three Wire Mode) SDA: USI Data Input (Two Wire Mode) PCINT5: Pin Change Interrupt 0, Source 5
PB6	DO: USI Data Output (Three Wire Mode) PCINT6: Pin Change Interrupt 0, Source 6
PB7	USCK: USI Clock (Three Wire Mode) SCL: USI Clock (Two Wire Mode) PCINT7: Pin Change Interrupt 0, Source 7

Figura 6: Alternate Functions of Port B

2.3.3. Funciones alternativas del puerto D

Port Pin	Alternate Function
PD0	RXD: UART Data Receiver PCINT11:Pin Change Interrupt 2, Source 11
PD1	TXD: UART Data Transmitter PCINT12:Pin Change Interrupt 2, Source 12
PD2	INT0: External Interrupt 0 Input XCK: USART Transfer Clock CKOUT: System Clock Output PCINT13:Pin Change Interrupt 2, Source 13
PD3	INT1: External Interrupt 1 Input PCINT14:Pin Change Interrupt 2, Source 14
PD4	T0: Timer/Counter0 Clock Source PCINT15:Pin Change Interrupt 2, Source 15
PD5	OC0B: Timer/Counter0 Compare Match B output T1: Timer/Counter1 Clock Source PCINT16:Pin Change Interrupt 2, Source 16
PD6	ICPI: Timer/Counter1 Input Capture Pin PCINT17:Pin Change Interrupt 2, Source 17

Figura 7: Alternate Functions of Port D

2.4. Diagrama de bloques

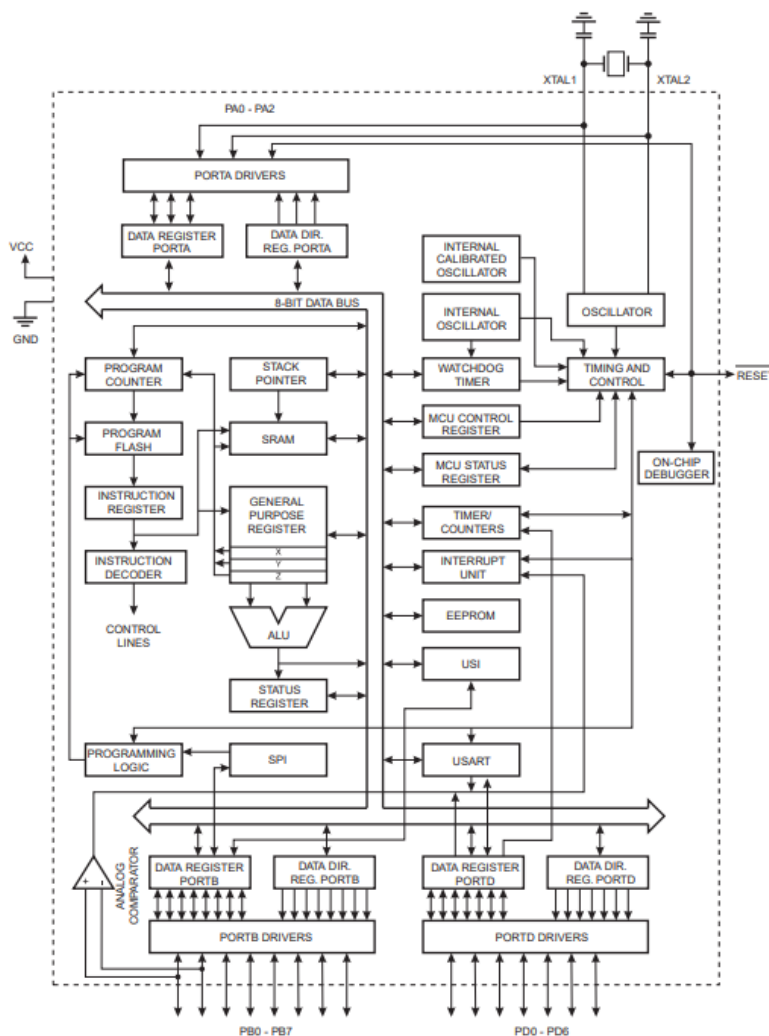


Figura 8: Diagrama de bloques, Microcontrolador ATtiny4313

2.5. Mapa de memoria de datos

	7	0	Addr.	
General Purpose Working Registers	R0		0x00	
	R1		0x01	
	R2		0x02	
	...			
	R13		0x0D	
	R14		0x0E	
	R15		0x0F	
	R16		0x10	
	R17		0x11	
	...			
	R26		0x1A	X-register Low Byte
	R27		0x1B	X-register High Byte
	R28		0x1C	Y-register Low Byte
	R29		0x1D	Y-register High Byte
	R30		0x1E	Z-register Low Byte
	R31		0x1F	Z-register High Byte

Figura 9: AVR CPU General Purpose Working Registers, Microcontrolador ATtiny4313

2.6. Características eléctricas

2.6.1. Absolute Maximum Ratings

Operating Temperature	-55°C to +125°C
Storage Temperature	-65°C to +150°C
Voltage on any Pin except $\overline{\text{RESET}}$ with respect to Ground	-0.5V to $V_{CC}+0.5V$
Voltage on $\overline{\text{RESET}}$ with respect to Ground.....	-0.5V to +13.0V
Maximum Operating Voltage	6.0V
DC Current per I/O Pin	40.0 mA
DC Current V_{CC} and GND Pins.....	200.0 mA

Figura 10: Absolute Maximum Ratings, Microcontrolador ATtiny4313

2.6.2. DC Characteristics

$T_A = -40^{\circ}\text{C}$ to 85°C , $V_{CC} = 1.8\text{V}$ to 5.5V (unless otherwise noted)

Symbol	Parameter	Condition	Min.	Typ.	Max.	Units
V_{IL}	Input Low Voltage except XTAL1 and RESET pin	$V_{CC} = 1.8\text{V} - 2.4\text{V}$ $V_{CC} = 2.4\text{V} - 5.5\text{V}$	-0.5		$0.2V_{CC}$ $0.3V_{CC}$	V
V_{IH}	Input High-voltage except XTAL1 and RESET pins	$V_{CC} = 1.8\text{V} - 2.4\text{V}$ $V_{CC} = 2.4\text{V} - 5.5\text{V}$	$0.7V_{CC}^{(1)}$ $0.6V_{CC}^{(1)}$		$V_{CC} + 0.5^{(2)}$	V
V_{IL1}	Input Low Voltage XTAL1 pin	$V_{CC} = 1.8\text{V} - 5.5\text{V}$	-0.5		$0.1V_{CC}$	V
V_{IH1}	Input High-voltage XTAL1 pin	$V_{CC} = 1.8\text{V} - 2.4\text{V}$ $V_{CC} = 2.4\text{V} - 5.5\text{V}$	$0.8V_{CC}^{(1)}$ $0.7V_{CC}^{(1)}$		$V_{CC} + 0.5^{(2)}$	V
V_{IL2}	Input Low Voltage RESET pin	$V_{CC} = 1.8\text{V} - 5.5\text{V}$	-0.5		$0.2V_{CC}$	V
V_{IH2}	Input High-voltage RESET pin	$V_{CC} = 1.8\text{V} - 5.5\text{V}$	$0.9V_{CC}^{(1)}$		$V_{CC} + 0.5^{(2)}$	V
V_{IL3}	Input Low Voltage RESET pin as I/O	$V_{CC} = 1.8\text{V} - 2.4\text{V}$ $V_{CC} = 2.4\text{V} - 5.5\text{V}$	-0.5		$0.2V_{CC}$ $0.3V_{CC}$	V
V_{IH3}	Input High-voltage RESET pin as I/O	$V_{CC} = 1.8\text{V} - 2.4\text{V}$ $V_{CC} = 2.4\text{V} - 5.5\text{V}$	$0.7V_{CC}^{(1)}$ $0.6V_{CC}^{(1)}$		$V_{CC} + 0.5^{(2)}$	V
V_{OL}	Output Low Voltage ⁽³⁾ (Except Reset Pin) ⁽⁵⁾	$I_{OL} = 20\text{ mA}$, $V_{CC} = 5\text{V}$ $I_{OL} = 10\text{ mA}$, $V_{CC} = 3\text{V}$			0.8 0.6	V V
V_{OH}	Output High-voltage ⁽⁴⁾ (Except Reset Pin) ⁽⁵⁾	$I_{OH} = -20\text{ mA}$, $V_{CC} = 5\text{V}$ $I_{OH} = -10\text{ mA}$, $V_{CC} = 3\text{V}$	4.2 2.4			V V
I_{IL}	Input Leakage Current I/O Pin	$V_{CC} = 5.5\text{V}$, pin low (absolute value)			$1^{(6)}$	μA
I_{IH}	Input Leakage Current I/O Pin	$V_{CC} = 5.5\text{V}$, pin high (absolute value)			$1^{(6)}$	μA
R_{RST}	Reset Pull-up Resistor		30		60	$k\Omega$
R_{pu}	I/O Pin Pull-up Resistor		20		50	$k\Omega$

Figura 11: DC Characteristics, Microcontrolador ATtiny4313

$T_A = -40^{\circ}\text{C}$ to 85°C , $V_{CC} = 1.8\text{V}$ to 5.5V (unless otherwise noted) (Continued)

Symbol	Parameter	Condition	Min.	Typ.	Max.	Units
I_{CC}	Power Supply Current	Active 1MHz, $V_{CC} = 2\text{V}^{(7)}$		0.2	0.55	mA
		Active 4MHz, $V_{CC} = 3\text{V}^{(7)}$		1.3	2.5	mA
		Active 8MHz, $V_{CC} = 5\text{V}^{(7)}$		3.9	7	mA
		Idle 1MHz, $V_{CC} = 2\text{V}^{(7)}$		0.03	0.15	mA
		Idle 4MHz, $V_{CC} = 3\text{V}^{(7)}$		0.25	0.6	mA
		Idle 8MHz, $V_{CC} = 5\text{V}^{(7)}$		1	2	mA
	Power-down mode	WDT enabled, $V_{CC} = 3\text{V}^{(8)}$		4	10	μA
		WDT disabled, $V_{CC} = 3\text{V}^{(8)}$		< 0.15	2	μA

Figura 12: DC Characteristics, Microcontrolador ATtiny4313

2.7. Periféricos

2.7.1. Registros

En el desarrollo del sistema de control de semáforos con el microcontrolador ATtiny4313, se emplearon diversos registros fundamentales que permitieron la configuración de los puertos, el manejo

de interrupciones y la temporización mediante un timer interno. A continuación, se describen de forma general los principales registros utilizados:

2.7.2. Registros de puertos (I/O)

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

Figura 13: Port Pin Configurations

- **DDRB y DDRA:** Se utilizan para configurar los pines del microcontrolador como entradas o salidas. En este caso, los pines conectados a LEDs fueron configurados como salidas, mientras que los pines conectados a los botones peatonales fueron configurados como entradas.
- **PORTB y PORTA:** Permiten controlar los niveles lógicos en los pines configurados como salida, lo cual fue utilizado para encender y apagar los LEDs de los semáforos vehiculares y peatonales.
- **PINB:** Se emplea para leer el estado actual de los botones conectados a los pines PB1 y PB2, determinando así cuándo un peatón solicita paso.

2.7.3. Registros de interrupciones

- **GIMSK y PCMSK:** Estos registros habilitan las interrupciones por cambio de estado en los pines del puerto B. Son claves para detectar de forma inmediata cuando un peatón presionaba un botón sin necesidad de consultar continuamente su estado (polling).

GIMSK – General Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x3B (0x5B)	INT1	INT0	PCIE0	PCIE2	PCIE1	–	–	–	GIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

Figura 14: GIMSK – General Interrupt Mask Register

PCMSK0 – Pin Change Mask Register 0

Bit	7	6	5	4	3	2	1	0	
0x20 (0x40)	PCINT7	PCINT6	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0	PCMSK0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figura 15: PCMSK0 – Pin Change Mask Register 0

- `sei()`: Es una instrucción que habilita globalmente el sistema de interrupciones del microcontrolador, permitiendo que las funciones asociadas a interrupciones se ejecuten cuando corresponde.

2.7.4. Registros del temporizador

- TCCR0A, TCCR0B y TCNT0: Se utilizan para configurar el Timer0 en modo normal, establecer el prescaler (para controlar la velocidad de conteo del temporizador) y reiniciar el contador al valor inicial.

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x30 (0x50)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figura 16: TCCR0A – Timer/Counter Control Register A

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x33 (0x53)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figura 17: TCCR0B – Timer/Counter Control Register B

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$\text{clk}_{I/O}$ /(No prescaling)
0	1	0	$\text{clk}_{I/O}/8$ (From prescaler)
0	1	1	$\text{clk}_{I/O}/64$ (From prescaler)
1	0	0	$\text{clk}_{I/O}/256$ (From prescaler)
1	0	1	$\text{clk}_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Figura 18: Clock Select Bit Description

TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
0x32 (0x52)	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figura 19: TCNT0 – Timer/Counter Register

- TIMSK: Permite habilitar y deshabilitar la interrupción por desbordamiento (overflow) del Timer0, lo cual fue utilizado para implementar retardos temporizados sin necesidad de funciones de espera bloqueante.

TIMSK – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x39 (0x59)	TOIE1	OCIE1A	OCIE1B	–	ICIE1	OCIE0B	TOIE0	OCIE0A	TIMSK
Read/Write	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figura 20: TIMSK – Timer/Counter Interrupt Mask Register

2.8. Componentes electrónicos complementarios

2.8.1. Leds

Los LEDs (Light Emitting Diodes) son dispositivos semiconductores que emiten luz cuando son polarizados directamente (ánodo positivo, cátodo negativo). Son ampliamente usados en sistemas electrónicos por su bajo consumo, tamaño reducido y larga vida útil. En los microcontroladores, los LEDs se utilizan comúnmente como indicadores visuales de estados del sistema (como semáforos, alarmas, actividad, etc.).

Para que un LED funcione correctamente, se debe respetar su tensión de umbral y limitar la corriente que lo atraviesa, típicamente con una resistencia en serie.

Led rojo: El LED rojo es un diodo emisor de luz que irradia en el espectro del color rojo, siendo uno de los tipos más comunes y utilizados en aplicaciones electrónicas debido a su bajo voltaje de operación y alta visibilidad. Se emplea habitualmente en sistemas de señalización, pantallas, indicadores de estado y semáforos, entre otros.

Su funcionamiento se basa en el principio de electroluminiscencia, donde al polarizarse directamente, los electrones se recombinan con huecos en el material semiconductor, liberando energía en forma de fotones de color rojo. Para lograr esta emisión, los LED rojos están fabricados con materiales como arseniuro de galio (GaAs) o arseniuro-fosfuro de galio (GaAsP), los cuales tienen una brecha de energía adecuada para emitir luz en esa región del espectro.

Los rangos de operación típicos para un LED rojo son:

- Tensión de operación (Forward Voltage): Aproximadamente entre 1.8V y 2.2V.
- Corriente de operación (Forward Current): Usualmente alrededor de 20 mA para una iluminación estándar.

- Potencia de salida: Dependiendo del tipo de encapsulado y la eficiencia del LED, puede oscilar entre 1 mW y 5 mW.
- Rango espectral: La longitud de onda típica del color rojo se encuentra entre 620 nm y 750 nm, dependiendo de la pureza del material y el diseño del LED.

Led amarillo: El LED amarillo es un dispositivo semiconductor que emite luz en el espectro visible correspondiente al color amarillo. Es ampliamente utilizado en aplicaciones que requieren señales visuales claras pero menos agresivas que las del rojo, como indicadores de advertencia, señalización vial, paneles informativos, y, por supuesto, semáforos.

El LED amarillo funciona, al igual que los demás LEDs, por electroluminiscencia, fenómeno en el que los electrones, al recombinarse con huecos en el material semiconductor, liberan energía en forma de fotones. Para emitir luz amarilla, estos LEDs están fabricados típicamente con compuestos como el arseniuro de galio-aluminio (AlGaAs) o fosfuro de galio-indio (InGaP).

Los rangos de operación típicos para un LED amarillo son:

- Tensión de operación (Forward Voltage): Aproximadamente entre 2.0V y 2.2V.
- Corriente de operación (Forward Current): Usualmente alrededor de 20 mA.
- Potencia de salida: Dependerá de la eficiencia del encapsulado, pero normalmente se encuentra entre 1.5 mW y 3 mW.
- Rango espectral: La longitud de onda del color amarillo se encuentra entre 585 nm y 595 nm, lo que le da su color característico visible y bien diferenciado del rojo o verde.

Led verde El LED verde es un diodo emisor de luz que opera dentro del espectro visible en la región del color verde. Se utiliza comúnmente en dispositivos electrónicos, indicadores de estado, sistemas de control, y señalización, incluyendo semáforos, donde representa condiciones de avance o autorización.

Los LEDs verdes están fabricados con materiales semiconductores como el fosfuro de galio (GaP), nitruro de galio-indio (InGaN) o combinaciones como GaN/InGaN, que permiten alcanzar longitudes de onda específicas dentro del rango del verde. Estos materiales permiten una buena eficiencia luminosa y una emisión clara y brillante.

Los rangos de operación típicos para un LED verde son:

- Tensión de operación (Forward Voltage): Aproximadamente entre 2.0V y 3.5V, dependiendo del tipo de tecnología usada. Los LEDs verdes tradicionales operan cerca de 2.0V–2.2V, mientras que los de alta intensidad pueden requerir hasta 3.2V–3.5V.
- Corriente de operación (Forward Current): Usualmente 20 mA.
- Potencia de salida: Puede variar entre 1.5 mW y 5 mW, dependiendo del encapsulado y la eficiencia del LED.
- Rango espectral: La longitud de onda de emisión se encuentra entre 520 nm y 570 nm, siendo el valor típico para un verde puro alrededor de 525–530 nm.

2.8.2. Botón con configuración Pull-Down

Un botón con configuración pull-down es un tipo de circuito de entrada digital comúnmente utilizado en microcontroladores para detectar el estado de un pulsador. Su función principal es asegurar que la entrada digital esté en un estado lógico definido (bajo o 0 lógico) cuando el botón no está siendo presionado.

¿Cómo funciona?: En esta configuración, un resistor pull-down (normalmente entre 10 k Ω y 100 k Ω) se conecta entre el pin de entrada del microcontrolador y **GND** (tierra). El botón se conecta entre el pin de entrada y V_{CC} (tensión positiva). Así, el circuito se comporta de la siguiente manera:

- Cuando el botón **NO** está presionado: El pin se encuentra conectado a tierra a través del resistor, por lo tanto, el microcontrolador lee un nivel lógico bajo (0).
- Cuando el botón **ESTÁ** presionado: El pin se conecta directamente a Vcc a través del botón, y el microcontrolador lee un nivel lógico alto (1).

Ventajas de la configuración pull-down:

- Asegura un estado lógico definido cuando el botón está abierto (no presionado).
- Evita lecturas erráticas o flotantes (ruido eléctrico) en la entrada del microcontrolador.
- Es fácil de implementar tanto en hardware como en simulaciones.

Aplicaciones:

- Detección de eventos de usuario (pulsaciones).
- Control de menús, selección de opciones, o activación de estados en sistemas digitales.
- Entrada segura y confiable para sistemas embebidos, como el caso del semáforo peatonal del laboratorio.

2.9. Diseño del circuito

A continuación se presenta el diseño final del circuito:

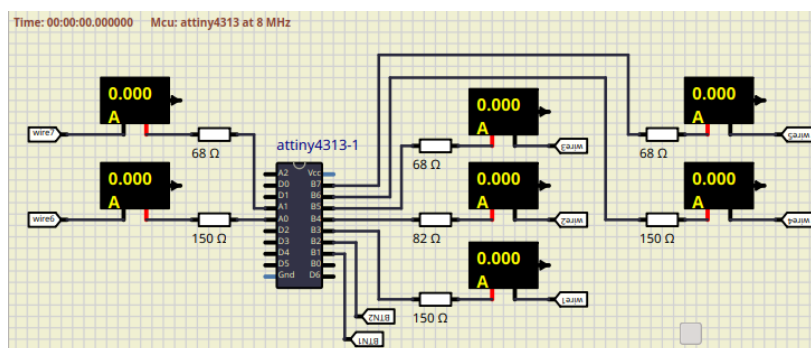


Figura 21: Diseño final, parte 1

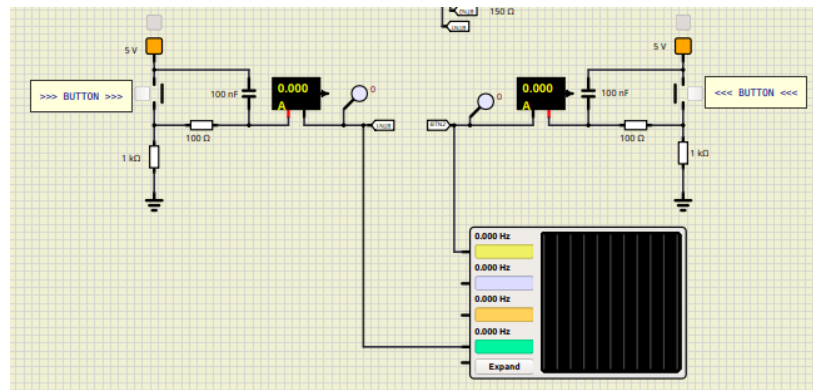


Figura 22: Diseño final, parte 2

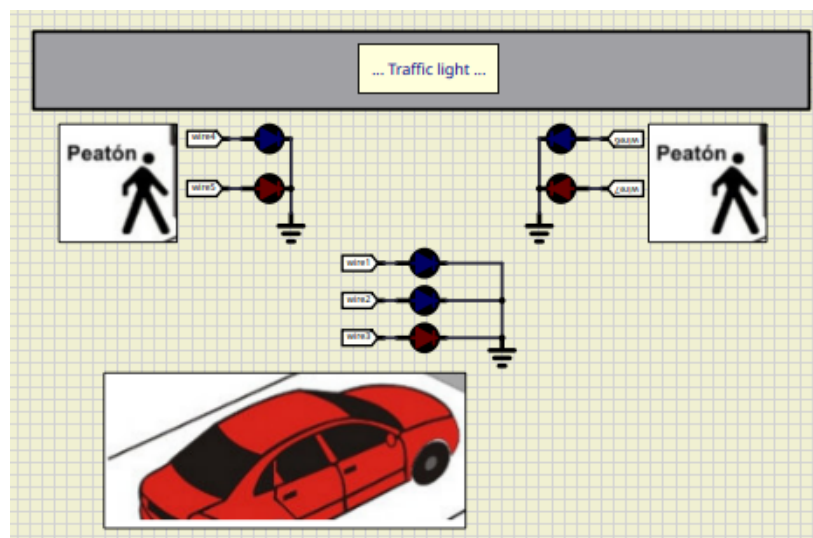


Figura 23: Diseño final, parte 3

2.9.1. Lista de componentes

2.9.2. Información adicional

GPIOs (General Purpose Input/Output): Los GPIOs son pines de propósito general que pueden configurarse como entradas o salidas. En un microcontrolador, los GPIOs permiten la interacción con el mundo exterior, ya sea recibiendo señales (como un botón) o enviándolas (como encender un LED). En este laboratorio:

- Se configuraron pines como salidas para controlar los LEDs de los semáforos.
- Se configuraron como entradas los pines donde se conectaron los botones de solicitud peatonal.

Interrupciones: Una interrupción es un mecanismo que permite al microcontrolador responder a eventos externos o internos de forma inmediata, deteniendo temporalmente la ejecución del programa principal para ejecutar una función específica llamada rutina de servicio a la interrupción (ISR). Esto

Tabla 1: Lista de componentes

Componente	Cantidad	Precio (CRC)
ATtiny4313	1	-
Pulsador (Push)	2	-
1k Ω	2	-
100 Ω	2	-
100nF	2	-
150 Ω	3	-
82 Ω	1	-
68 Ω	3	-
Led rojo	3	-
Led amarillo	1	-
Led verde	3	-

es útil para responder rápidamente a eventos como la presión de un botón, sin necesidad de revisar continuamente su estado. En este laboratorio, se utilizaron interrupciones por cambio de pin (pin change interrupt) para detectar cuándo un peatón solicita paso.

Los vectores de interrupción del ATtiny2313A/4313 se describen a continuación.

Vector No.	Program Address	Label	Interrupt Source
1	0x0000	RESET	External Pin, Power-on Reset, Brown-out Reset, and Watchdog Reset
2	0x0001	INT0	External Interrupt Request 0
3	0x0002	INT1	External Interrupt Request 1
4	0x0003	TIMER1 CAPT	Timer/Counter1 Capture Event
5	0x0004	TIMER1 COMPA	Timer/Counter1 Compare Match A
6	0x0005	TIMER1 OVF	Timer/Counter1 Overflow
7	0x0006	TIMER0 OVF	Timer/Counter0 Overflow
8	0x0007	USART0, RX	USART0, Rx Complete
9	0x0008	USART0, UDRE	USART0 Data Register Empty
10	0x0009	USART0, TX	USART0, Tx Complete
11	0x000A	ANALOG COMP	Analog Comparator
12	0x000B	PCINT0	Pin Change Interrupt Request 0
13	0x000C	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x000D	TIMER0 COMPA	Timer/Counter0 Compare Match A
15	0x000E	TIMER0 COMPB	Timer/Counter0 Compare Match B
16	0x000F	USI START	USI Start Condition
17	0x0010	USI OVERFLOW	USI Overflow
18	0x0011	EE READY	EEPROM Ready
19	0x0012	WDT OVERFLOW	Watchdog Timer Overflow
20	0x0013	PCINT1	Pin Change Interrupt Request 1
21	0x0014	PCINT2	Pin Change Interrupt Request 2

Figura 24: Reset and Interrupt Vectors

Timers: Los timers son periféricos que permiten medir el tiempo o generar retardos sin bloquear

la ejecución del programa. Funcionan mediante un contador que incrementa a intervalos regulares, determinados por un reloj interno y un prescaler. Cuando el contador llega a su valor máximo (overflow), puede generar una interrupción. En este laboratorio, el Timer0 fue configurado en modo normal con una interrupción por desbordamiento, y se utilizó para generar retardos temporizados para los cambios de estado del semáforo.

Máquina de Estados Finita (FSM): Una máquina de estados finita es un modelo de programación que divide el comportamiento de un sistema en **estados**. Cada estado representa una situación específica, y las transiciones entre estados se dan según eventos o condiciones. Este enfoque mejora la organización del código y permite implementar comportamientos secuenciales complejos. En el sistema de semáforo, se diseñaron tres estados:

1. Paso de vehículos
2. Waiting (Esperando)
3. Paso de peatones

Cada estado controla los LEDs correspondientes y define qué condiciones deben cumplirse para pasar al siguiente.

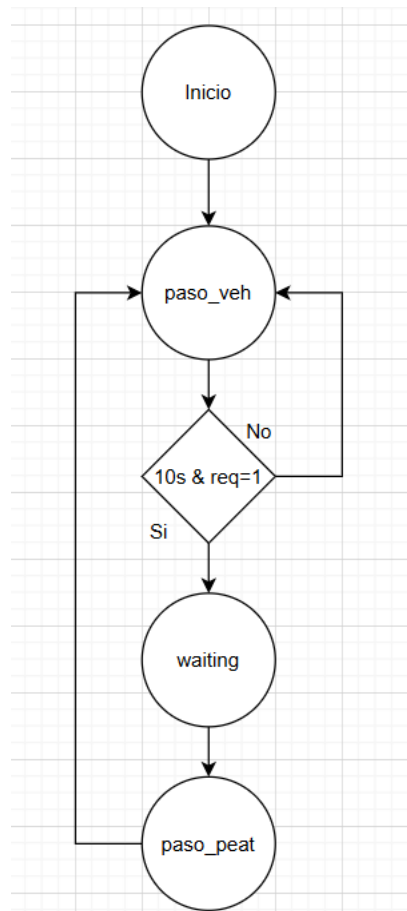


Figura 25: Diagrama de estados de la FSM.

3. Desarrollo

3.1. Análisis del firmware

3.1.1. Configuración General y Variables

```
#include <avr/io.h>
#include <avr/interrupt.h> // Manejo de interrupciones
```

Estas líneas incluyen las bibliotecas necesarias para usar periféricos del microcontrolador y gestionar interrupciones.

```
/* Prototipos de funciones */
void traffic_light();
void FSM();
void start_delay(uint16_t);
```

Se declaran funciones para controlar el sistema de semáforos (`traffic_light`), manejar la máquina de estados (`FSM`) y generar retardos con temporizador (`start_delay`).

```
/* Variables globales */
volatile uint8_t request = 0;
unsigned int state = 0, next_state = 0;
volatile uint8_t en_delay = 0;
volatile uint16_t times = 0;
volatile uint16_t target_times = 0;
volatile uint8_t delay_done = 0;
```

Variables para:

- `request`: Indica si hubo solicitud de cruce peatonal.
- `state` y `next_state`: Estados actuales y próximos de la FSM.
- `en_delay`: Activa el conteo del temporizador.
- `times`: Conteo de desbordamientos (overflows).
- `target_times`: Valor objetivo de overflows a esperar.
- `delay_done`: Marca cuándo terminó el retardo.

3.1.2. Definición de Estados para la FSM

```
#define paso_vehiculos 0
#define waiting        1
#define paso_peatones  2
```

Se definen nombres legibles para los estados de la máquina de estados finita.

3.1.3. Función principal (main)

```
int main(void) {  
    traffic_light();  
}
```

Llama directamente a la lógica principal del sistema.

3.1.4. Función principal del semáforo

```
void traffic_light(){  
    DDRB = 0xF9; // Configura PB0 como entrada, PB1 y PB2 como entradas para botones  
    DDRA = 0x03; // PA0 y PA1 como salidas (peatones derechos)  
  
    GIMSK |= (1 << PCIE0); // Habilita interrupción por cambio de pin (PCINT0)  
    PCMSK |= 0x06;         // Habilita interrupciones en PB1 y PB2  
  
    TCCR0A |= 0x00;         // Timer0 en modo normal  
    TCCR0B |= 0x04;         // Prescaler de 256 → f_timer = 8MHz / 256 = 31.25kHz  
    TCNT0 = 0x00;          // Contador del Timer en 0  
  
    sei();                  // Habilita interrupciones globales  
  
    next_state = paso_vehiculos;  
  
    while (1) {  
        FSM();  
    }  
}
```

Se configuran:

- Puertos de entrada/salida
- Interrupciones externas por botones
- Temporizador (Timer0) en modo normal con prescaler de 256
- Se entra en un bucle infinito con la máquina de estados.

3.1.5. Máquina de estados finita (FSM)

```
void FSM(){  
    state = next_state;  
    switch(state){
```

La FSM tiene 3 casos principales:

3.1.6. Estado paso_vehiculos (vehículos en verde)

```
case paso_vehiculos:
    PORTB = (1 << PB5) | (1 << PB6); // Verde vehículo(PB5) + Rojo peatones izq(PB6)
    PORTA = (1 << PA0);                // Rojo peatones der (PA0)
    start_delay(1221);                  // 10s de retardo → 1221 overflows
    while (!delay_done);                // Espera sin bloquear el sistema (se usa ISR)
    if (request) {
        request = 0;
        next_state = waiting;
    } else {
        next_state = paso_vehiculos;
    }
break;
```

3.1.7. Estado waiting (vehículos en amarillo)

```
case waiting:
    PORTB = (1 << PB4) | (1 << PB6); // Amarillo vehículo(PB4)+Rojo peatones izq(PB6)
    PORTA = (1 << PA0);                // Rojo peatones der (PA0)
    start_delay(366);                  // 3s → 366 overflows
    while (!delay_done);
    next_state = paso_peatones;
break;
```

3.1.8. Estado paso_peatones (peatones en verde)

```
case paso_peatones:
    PORTB = (1 << PB3);                // Rojo vehículo (PB3)
    PORTB |= (1 << PB7);                // Verde peatones izq (PB7)
    PORTA = (1 << PA1);                // Verde peatones der (PA1)
    start_delay(1221);                  // 10s
    while (!delay_done);
    next_state = paso_vehiculos;
    PCMSK |= (1 << PCINT1) | (1 << PCINT2); // Reactivar interrupciones en botones
break;
```

3.1.9. Función de retardo con temporizador

```
void start_delay(uint16_t overflows){
    en_delay = 1;
    delay_done = 0;
    times = 0;
    target_times = overflows;
    TIMSK |= (1 << TOIE0);             // Habilita interrupción por desbordamiento de Timer0
}
```

Activa la ISR del temporizador y prepara el conteo de overflows.

3.1.10. Interrupción por botón (ISR externa)

```
ISR(PCINT0_vect){
    if ((PINB & (1 << PB1)) || (PINB & (1 << PB2))){
        request = 1;
        PCMSK &= ~((1 << PCINT1) | (1 << PCINT2)); // Desactiva interrupciones
    }
}
```

Cuando se presiona un botón (PB1 o PB2), se activa la interrupción de cambio de pin:

- Se marca `request = 1` para indicar que se solicitó cruce peatonal.
- e desactivan las interrupciones para evitar múltiples solicitudes hasta que se reactiven después.

3.1.11. Interrupción por Timer Overflow

```
ISR(TIMERO_OVF_vect){
    if (en_delay){
        times++;
        if (times >= target_times){
            en_delay = 0;
            delay_done = 1;
            TIMSK &= ~(1 << TOIE0); // Desactiva la interrupción del temporizador
        }
    }
}
```

Cada vez que el timer se desborda ($0xFF \rightarrow 0x00$), entra esta ISR y:

- Incrementa el contador `times`.
- Si se alcanzó el número deseado de overflows (`target_times`), termina el retardo y desactiva la interrupción.

3.1.12. Cálculo de Overflows

Para calcular cuántos overflows se necesitan para un cierto tiempo:

Fórmula:

$$T_{overflow} = \frac{256 \cdot Prescaler}{F_{CPU}}$$

$$Overflows = \frac{T_{deseado}}{T_{overflow}}$$

con:

- $F_{CPU} = 8 \text{ MHz}$
- Prescaler = 256

$$T_{overflows} = \frac{256 \cdot 256}{8000000} \approx 8,192 \text{ ms}$$

Entonces:

- Para **10 segundos** $\rightarrow \frac{10}{0,008192} \approx 1221 \text{ overflows}$
- Para **3 segundos** $\rightarrow \frac{3}{0,008192} \approx 366 \text{ overflows}$

3.2. Análisis electrónico

3.2.1. Implementación de los botones

Para la implementación de los botones, se opta por una configuración de pull-down, con una valor de resistencia de $1\text{k}\Omega$, y para tratar el efecto rebote se conecta un capacitor de 100nF , con lo cual se forma una red RC, de la cual se obtiene una constante de tiempo $\tau = 100 \text{ micro segundos}$ ($100 \mu\text{s}$).

Analizando mas a fondo el funcionamiento, cuando el pulsador no es presionado, se observa que la tensión en la entrada del pin es 0 V , y esto tiene sentido por ser una configuración de pull-down,

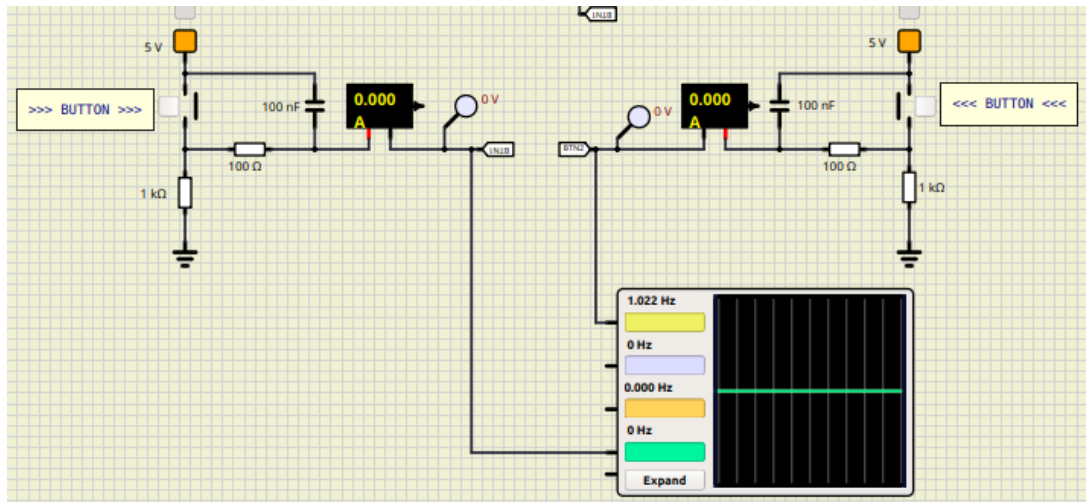


Figura 26: Botón off

ahora bien cuando alguno de los botones es presionado, de acuerdo al diseño del circuito, se tiene que hay un cambio de estado lógico, y la tensión a la entrada del pin es un 1 lógico que son los 5 V .

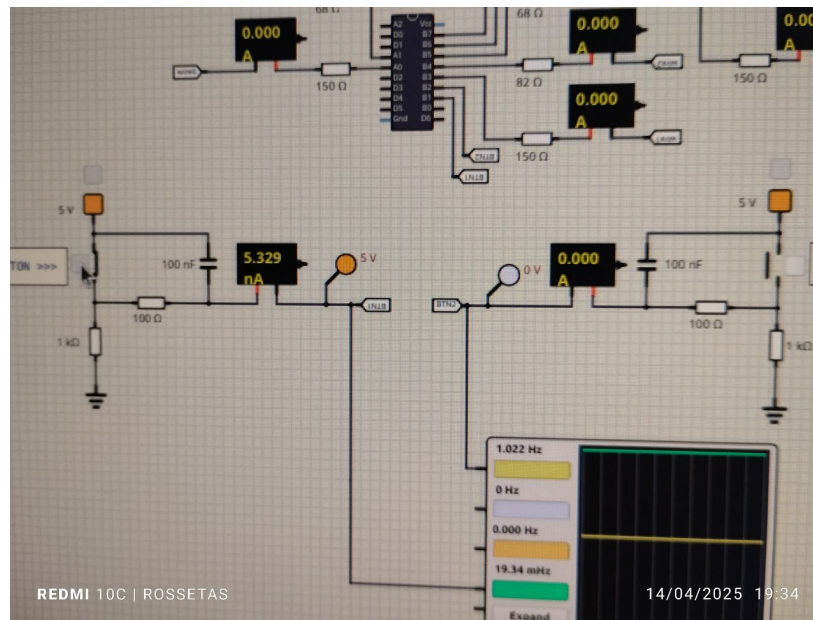


Figura 27: Botón1 on
foto tomada con el celular

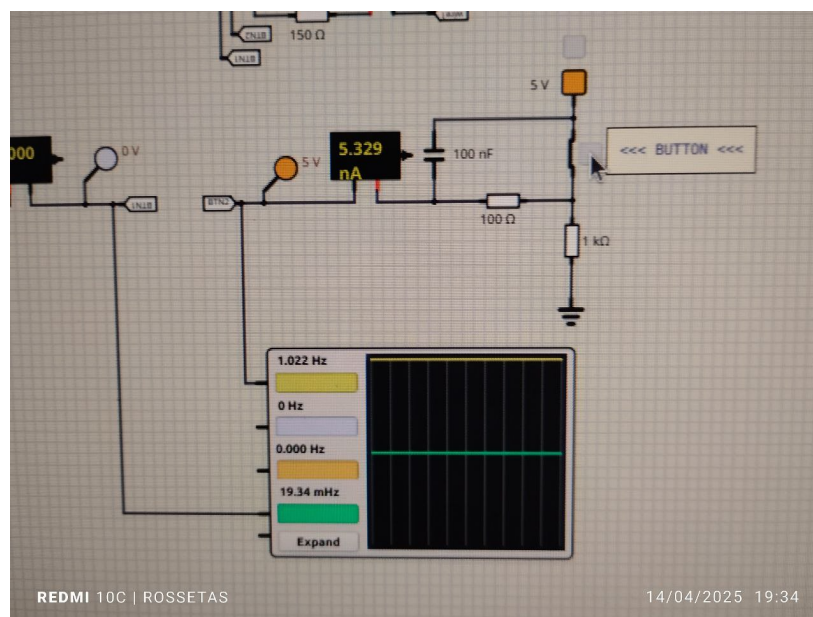


Figura 28: Botón2 on
foto tomada con el celular

cabe destacar que la corriente de entrada al pin es despreciable, es una corriente muy baja que no afecta el desempeño del circuito, y por lo tanto se tiene una correcta implementación de pulsadores (botones) para el funcionamiento del circuito, cabe recalcar también que los botones funcionan como la entrada del sistema, por lo que se demuestra de igual manera que el funcionamiento es óptimo y de acuerdo a los requisitos que se plantean en el sistema.

3.2.2. Iluminación de los Leds

En cuanto a la iluminación de los leds, que tienen el rol de mostrar la salida del sistema se tienen que tener las siguientes consideraciones, por tratarse de un sistema simplificado se utilizan leds, para simular las distintas etapas presentes en el funcionamiento de un cruce peatonal, por lo tanto, para mostrar un poco más de veracidad en los cálculos, se utilizan componentes físicos dentro de la simulación, por lo tanto, datos como la tensión de umbral y corriente máxima, son tomados de las hojas de fabricantes presentes en el directorio **etc** del laboratorio, entonces se tiene lo siguiente:

3.2.3. Led rojo

De acuerdo a la hoja de fabricante del led rojo se tiene que:

Electrical Optical Characteristics at $T_a = 25^\circ\text{C}$

Parameter	Symbol	Minimum	Typical	Maximum	Unit	Test Condition
Luminous Intensity	I_v		40		mcd	$I_f = 20\text{ mA}$ (Note 1)
Viewing Angle	$2\theta_{1/2}$		25		Deg	(Note 2)
Peak Emission Wavelength	λ_p		640		nm	$I_f = 20\text{ mA}$
Dominant Wavelength	λ_d		635		nm	$I_f = 20\text{ mA}$ (Note 3)
Spectral Line Half-Width	$\Delta\lambda$		25		nm	$I_f = 20\text{ mA}$
Forward Voltage	V_f		2	2.5	V	$I_f = 20\text{ mA}$
Reverse Current	I_R	-	-	100	μA	$V_R = 5\text{ V}$

Figura 29: Datasheet led rojo

Como se observa en las características eléctricas del led rojo, se tiene que el voltaje directo (forward voltage) típico es de 2 V a una corriente de 20 mA, por lo tanto para los leds rojo tenemos la siguiente asignación:



Figura 30: Led rojo

y para los led rojos tenemos que se conectan resistencias de 150Ω para limitar la corriente.

3.2.4. Led amarillo

De acuerdo a la hoja de fabricante del led amarillo se tiene que:

Electrical/Optical Characteristics at T _a = 25°C						
Parameter	Symbol	Minimum	Typical	Maximum	Unit	Test
Luminous Intensity	I _v	-	9.8	-	mcd	I _f = 20mA
Peak Emission Wavelength	λ _P	-	587	595	nm	Measurement at Peak I _f = 20mA-
Dominant Wavelength	λ _D	-	590	-		
Operating Voltage	V _{od}	3	5	10	V	-
Blinking Frequency	F _{blk}	2	2.4	2.8	Hz	-
Reverse Current	I _R	-	-	100	μA	V _R = 5V

Figura 31: Datasheet led amarillo

de igual forma, como se observa en las características eléctricas del led amarillo, se tiene que el voltaje directo (forward voltage) minimo es de 3 V a una corriente de 20 mA, por la tanto para los leds amarillos tenemos la siguiente asignación:

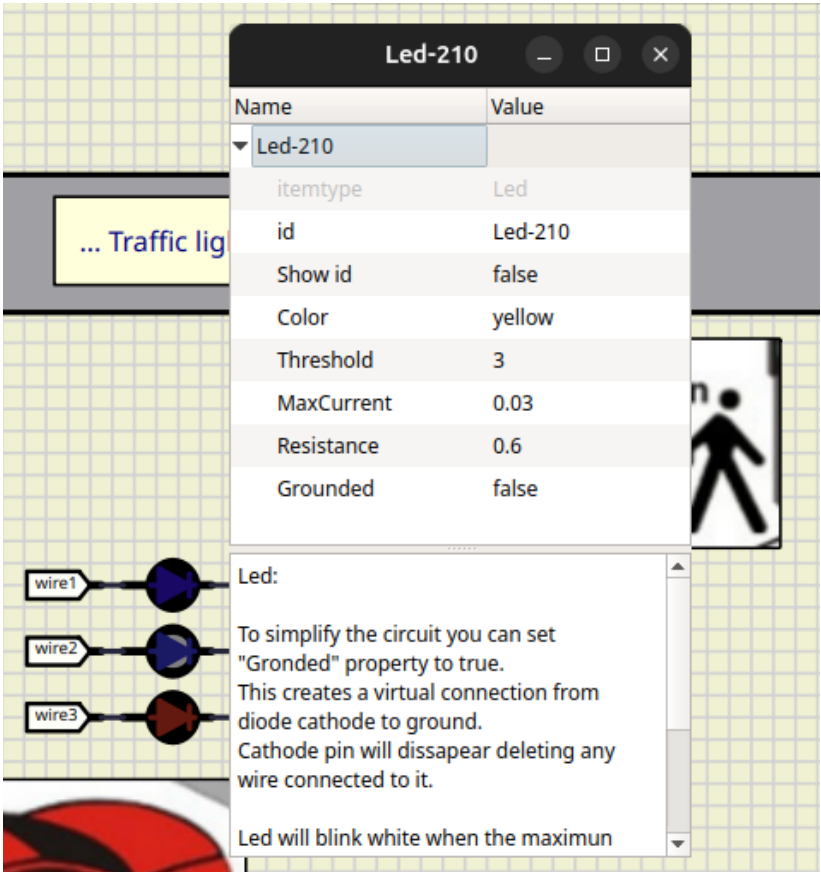


Figura 32: Led amarillo

y para los led amarillo tenemos que se conectan resistencias de 82Ω para limitar la corriente.

3.2.5. Led verde

Finalmente, de acuerdo a la hoja de fabricante del led verde se tiene que:

Electrical and Optical Characteristics at TA = 25°C

LED Chip			Lens Colours	Dominant Wavelength (nm) at 20mA	Luminous Intensity (mcd) at 20mA		Forward Voltage (V) at 20mA		Viewing Angle 2θ ^{1/2} (°)
Material	Emitted Colours	Brightness			Minimum	Typical	Typical	Maximum	
InGaN / Sapphire	True Green	Mega	Water Clear	520	19,500	45,000	3.2	4	15

Figura 33: Datasheet led verde

de ultimo, como se observa en las características eléctricas del led verde, se tiene que el voltaje directo (forward voltage) tipico es de 3.2 V a una corriente de 20 mA, por la tanto para los leds verdes tenemos la siguiente asignación:

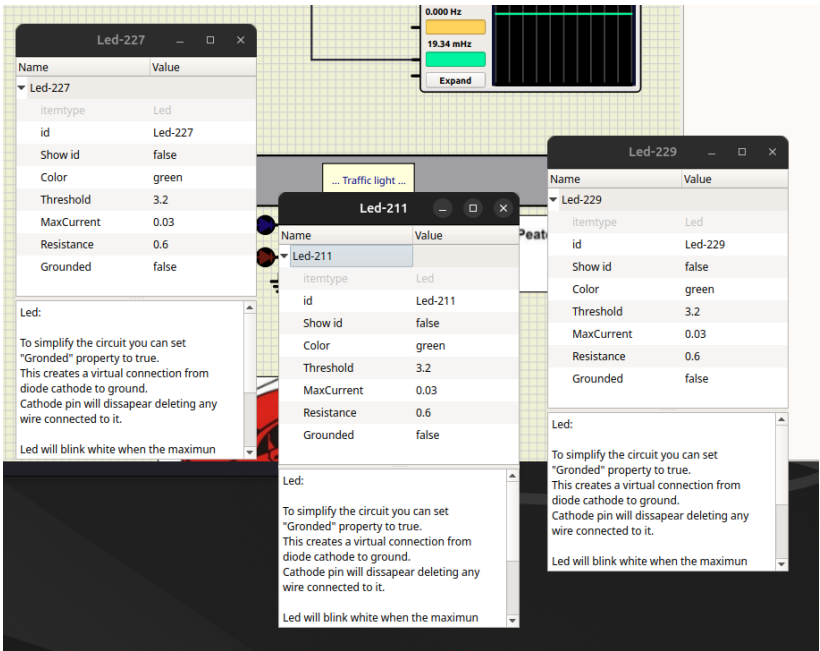


Figura 34: Led verde

y para los leds verdes tenemos que se conectan resistencias de 68Ω para limitar la corriente.

3.2.6. Configuración del reloj en el MCU

El ATtiny4313 cuenta con varios sistemas de reloj que determinan la velocidad a la que opera el microcontrolador. El sistema de reloj es fundamental porque controla el ritmo de ejecución de instrucciones, temporizadores y periféricos internos.

Tipos de fuentes de reloj que puede usar el ATtiny4313:

- Reloj interno RC de 8 MHz (el más común y el que se usa en el laboratorio).
- Reloj interno RC de 128 kHz

- Oscilador externo (cristal, resonador cerámico o reloj externo).
- Reloj de baja frecuencia externo.

En resumen, el ATtiny4313 en el laboratorio opera con su oscilador interno RC de 8 MHz, lo cual proporcionó una base de tiempo precisa y confiable para controlar los semáforos mediante temporizadores. Esta configuración permite usar retardos cronometrados con el Timer0 y simplifica el diseño al no requerir componentes externos de reloj.

Entonces para hacer valido el reloj del MCU se tienen las siguientes modificaciones:

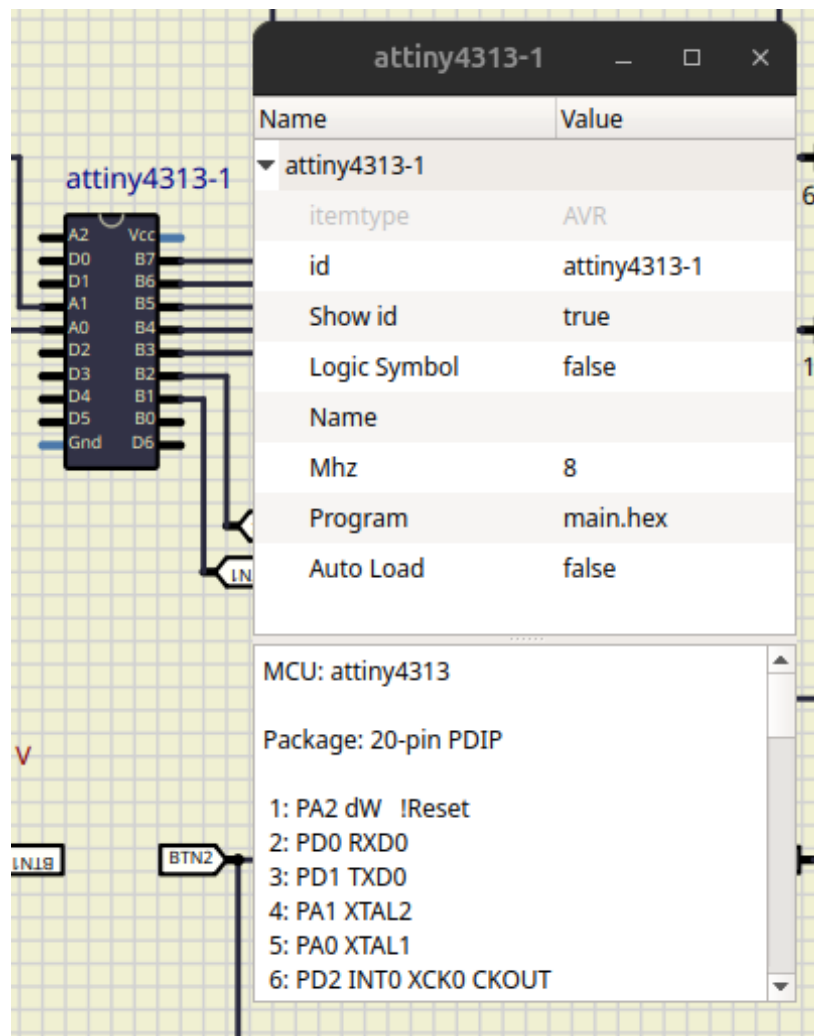


Figura 35: Configuración del reloj

y esto también se modifica en el archivo **Makefile** con la siguiente línea:

```
F_CPU=8000000 # @8MHz
```

3.3. Análisis de resultados

En este apartado de análisis de resultados, se opta por hacer un análisis por etapas del funcionamiento en tiempo de ejecución, por lo tanto, se tiene lo siguiente:

3.3.1. Paso de vehículos

Como se observa en la siguiente figura, el estado de paso de vehículos muestra un correcto funcionamiento, en donde se aprecia que el led verde del semáforo vehicular se encuentra activo al igual que los leds rojos del semáforo peatonal, presentando un funcionamiento adecuado de acuerdo a los requisitos del sistema.

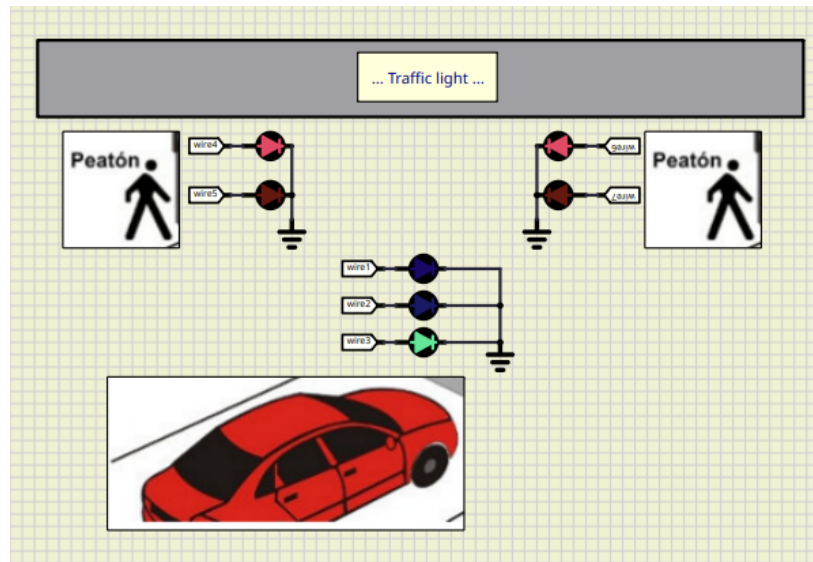


Figura 36: Funcionamiento estado paso de vehículos

Una vez visto el funcionamiento en tiempo de ejecución se muestra una imagen de las características eléctricas de los componentes, en donde se puede apreciar que ninguna de las corrientes supera los 20 mA, que se establecen como umbral máximo para un funcionamiento adecuado y se tiene certeza que no va a ocasionar fallos.

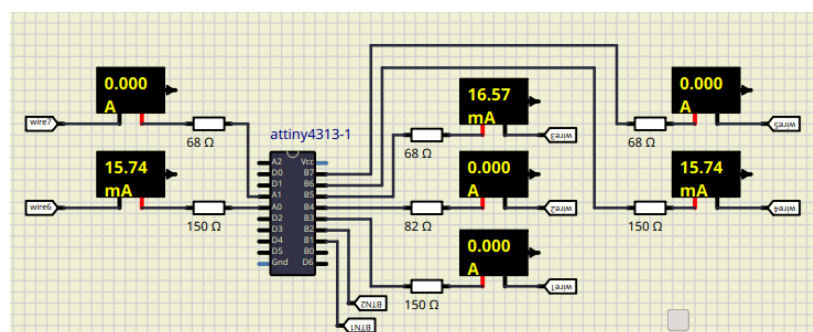


Figura 37: Características de los componentes, estado paso de vehículos

3.3.2. Waiting (Modo de espera)

De igual forma, para el modo de espera, se tiene que el led amarillo del semáforo vehicular se torna en activo, y los leds rojos del semáforo peatonal permanece en activo, y cumple con lo esperado.

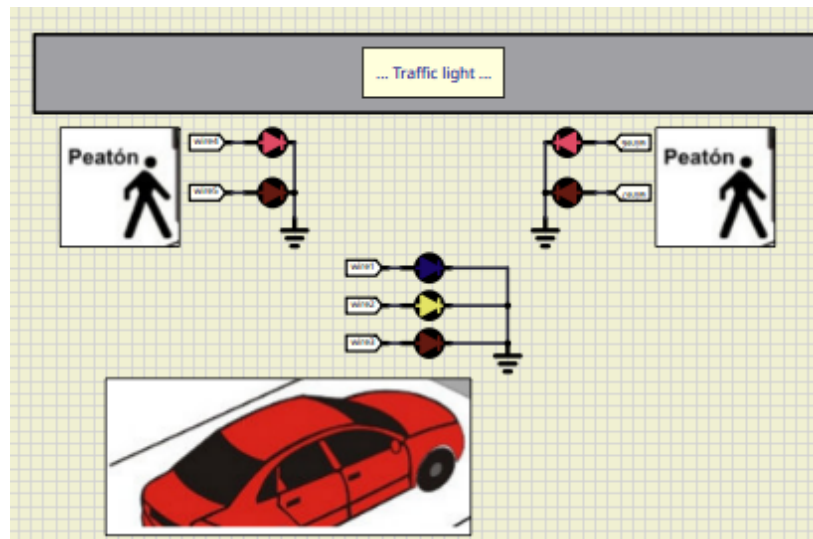


Figura 38: Funcionamiento estado waiting

En cuando a las características eléctricas en el modo de espera, no hay mucho cambio, mas la activación del led amarillo y que de igual manera tiene una corriente de 16,31 mA que se encuentra dentro del rango establecido.

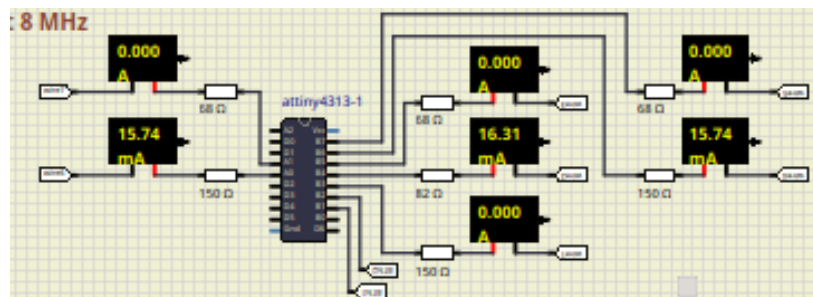


Figura 39: Características de los componentes, estado waiting

3.3.3. Paso de peatones

Finalmente, el estado de peatones tiene un funcionamiento correcto, en donde se contempla de acuerdo a la salida que se muestra en la figura, que el led rojo del semáforo vehicular se activa mientras que los leds verdes del semáforo peatonal pasan a estar activas de igual forma.

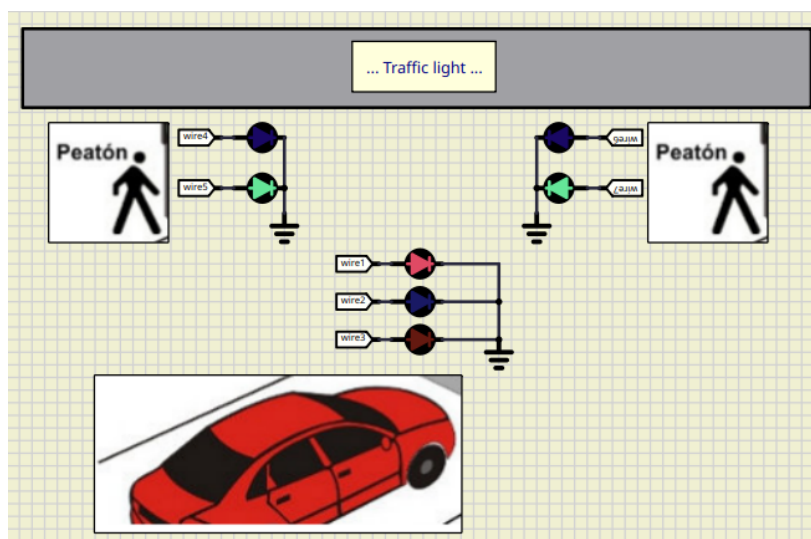


Figura 40: Funcionamiento estado paso de peatones

y por consiguiente, se muestran las características eléctricas de los componentes que corresponden al estado de paso de peatones, las corrientes como se observa siempre están dentro del rango establecido, que es no superar los 20 mA por cuestiones de seguridad.

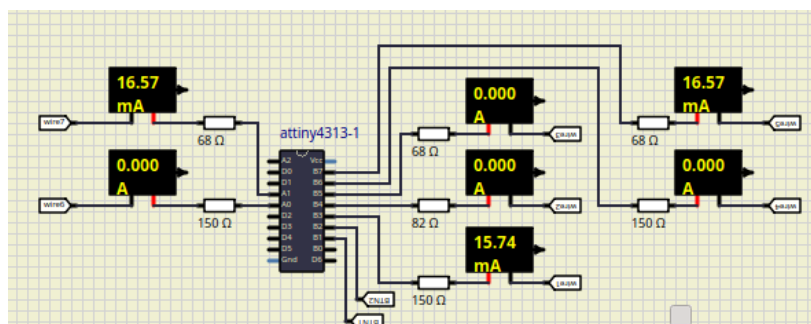


Figura 41: Características de los componentes, estado paso de peatones

4. Conclusiones y Recomendaciones

4.1. Conclusiones

Durante el desarrollo del laboratorio, se logró implementar correctamente un sistema de cruce de semáforos controlado por el microcontrolador ATtiny4313, haciendo uso de botones para solicitudes peatonales, LEDs para simular luces vehiculares y peatonales, e interrupciones para una respuesta eficiente a eventos externos.

Entre los principales logros se destacan:

- La correcta configuración y uso del Timer0 para generar retardos temporizados, utilizando el reloj interno de 8 MHz y prescaler de 256.
- La implementación de interrupciones externas (PCINT) para capturar de manera eficiente la solicitud de cruce peatonal sin bloquear la ejecución principal.
- El diseño de una máquina de estados (FSM) clara y funcional que gestiona las transiciones entre los estados del sistema: paso de vehículos, espera y paso de peatones.
- La comprensión del funcionamiento del reloj interno del microcontrolador y su impacto directo en los cálculos de tiempo y sincronización del sistema.

Este laboratorio permitió aplicar conceptos clave de sistemas embebidos como temporización, interrupciones, control digital, lógica secuencial y diseño modular de código.

4.2. Recomendaciones

- Siempre tener en cuenta la resistencia de protección para los Leds.
- Siempre tener en cuenta las características eléctricas de los componentes, es fundamental conocer acerca de eso.
- Se recomienda verificar cuidadosamente los cálculos de retardos basados en overflows del temporizador, asegurándose de considerar correctamente el prescaler y la frecuencia de reloj, para evitar tiempos erróneos.
- Utilizar nombres más descriptivos para las variables y constantes en el código puede facilitar la legibilidad y el mantenimiento del mismo.

Bibliografía

1. Material de clase
2. datasheet ATtiny4313
3. AVR Libc Documentation. (2021). <avr/interrupt.h><https://www.nongnu.org/avr-libc/user-manual/modules.html>
4. datasheet Led rojo
5. datasheet Led amarillo
6. datasheet Led verde

5. Apéndices

5.1. Repositorio Git

Repositorio GitHub: <https://github.com/Rossetas/MCU/tree/lab2>